SAE INSTITUTE

Error Handling in Games

Elias Farhan

University of Hertfordshire UH

What are we doing today?

- How a function reports that something **went wrong**
- Classic **error codes** and `std::error_code`
- Errors as **values**: `std::optional`, `std::expected`
- **Exceptions**: how they work, and why games often **ban** them
- Ways to **leave a program**: `std::terminate`, `std::abort`, `std::exit`
- **noexcept**, then catching **bugs** with `assert`, **C++26 contracts**, and `static_assert`

What does “wrong” even mean?

Two kinds of wrong

Before picking a tool, decide which problem you have. They are **not** the same.

- **Errors**: expected at runtime, from the outside world, like a missing file or a bad packet
- **Bugs**: broken assumptions in *your* code, like a null pointer or an out-of-range index
- Errors are **handled and recovered**; bugs are **fixed before shipping**

What goes wrong in a frame?

A running game constantly touches things that can **fail** for reasons you do not control.

- An asset fails to load or is **corrupt**
- A save file is from an **older** version
- A network packet is **malformed**
- A config value is **out of range**

SAE INSTITUTE

How can a function signal failure?

One function, several conventions. Each has a different cost and ergonomics.

- Return an **error code**, write the result to an out-parameter
- Return an **optional** or **expected** value
- **Throw** an exception
- **Abort** the program (it was a bug, not an error)

Error codes

The C-style approach

Return a status, hand the real result back through an **out-parameter**.

```
int load_texture(const char* path, Texture* out);   // 0 = ok, non-zero = err

Texture tex;
if (load_texture("hero.png", &tex) != 0) {
    // handle failure
}
```

Error codes are easy to ignore

Nothing forces the caller to **look** at the return value.

```
load_texture("hero.png", &tex);   // return value silently dropped
use(tex);                         // tex may be garbage
```

Naming them with an enum

A scoped enum makes each failure **self-documenting** instead of a magic number.

```
enum class LoadError { none, not_found, bad_format, out_of_memory };

LoadError load_texture(const char* path, Texture& out);
```

std::error_code

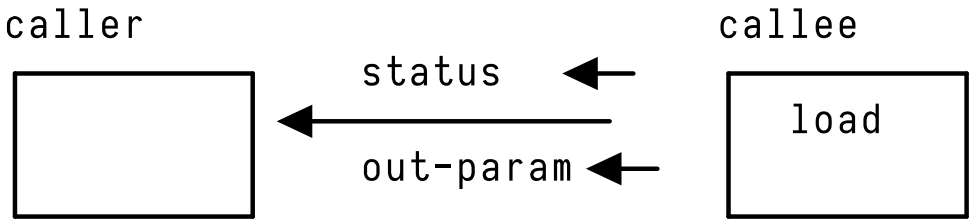
The standard's portable error type: an **integer plus a category** that explains it.

```
#include <system_error>

std::error_code ec;
auto bytes = read_file("level.bin", ec);
if (ec) {
    log(ec.message());        // human-readable, e.g. "No such file or
}
```

The out-parameter problem

The value and the error travel through **two different channels**, so they can disagree.



`value valid only if status says so, by hand`

What if there was a way...

to return **either** a value **or** an error, in a single object the caller cannot ignore?

Errors as values

std::optional

A value that **might not be there**. Either it holds a `T`, or it holds nothing.

```
#include <optional>

std::optional< Entity > find_entity(EntityId id);

if (auto e = find_entity(id)) {
    use(*e);                    // present
} else {
    // absent
}
```

optional says “no”, not “why”

It can tell you a value is **missing**, but never the **reason** it failed.

```
std::optional< Texture > load(const char* path);
// not_found? bad_format? out_of_memory? optional cannot say
```

std::expected

Since C++23: holds either a value or an error, in one type.

```
#include <expected>

std::expected< Texture, LoadError > load_texture(const char* path);
```

Reading an expected

Check first, then reach for the value **or** the error, never both.

```
auto result = load_texture("hero.png");
if (result) {
    use(*result);                // or result.value()
} else {
    log(to_string(result.error()));
}
```

Returning success or failure

Build a value directly, or return a `std::unexpected` carrying the error.

```
std::expected< Texture, LoadError > load_texture(const char* path) {
    if (!exists(path))   return std::unexpected(LoadError::not_found);
    Texture t = decode(path);
    return t;                        // success
}
```

Chaining without if-checks

Monadic operations let errors **short-circuit** through a pipeline.

```
auto hp = load_texture("hero.png")
    .and_then(upload_to_gpu)      // runs only on success
    .transform(make_sprite)       // maps the value
    .or_else(use_placeholder);    // runs only on error
```

What if there was a way...

to make a function that **cannot** return normally just **jump out**, past every caller in between?

Exceptions

throw, try, catch

An exception **abandons** the normal path and unwinds until someone **catches** it.

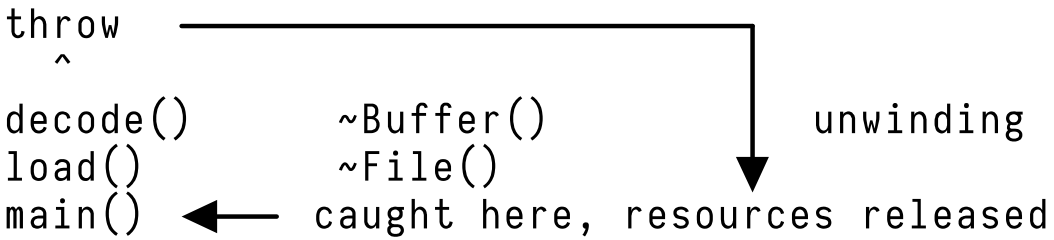
```
Texture load_texture(const char* path) {
    if (!exists(path)) throw std::runtime_error("not found");
    return decode(path);
}

try             { auto t = load_texture("hero.png"); }
catch (const std::exception& e) { log(e.what()); }
```

Stack unwinding runs destructors

As the exception travels up, every local object on the way is **destroyed**.

RAII still holds.



Exceptions clean up the happy path

The success code reads top-to-bottom, with **no error checks** interleaved.

```
auto tex    = load_texture("hero.png");   // throws on failure
auto sprite = make_sprite(tex);
auto id     = upload_to_gpu(sprite);      // no `if (err)` between steps
```

Exceptions are not free

The mechanism costs something whether or not you ever `throw`.

- Larger **binary**: unwinding tables for every function
- A `throw` is **slow**, orders of magnitude past a return
- Timing becomes **unpredictable**, bad for a fixed frame budget
- Harder to reason about every possible **exit** path

Why games often disable them

Performance and platform constraints push many engines to turn exceptions **off**.

- Console SDKs and hot loops want **predictable** cost
- Many engines ship with `-fno-exceptions`
- Alternative libraries (e.g. **EASTL**) assume no exceptions
- Determinism matters for **replays** and **netcode**

Building without exceptions

Disable them at the compiler, and a stray `throw` **terminates** instead of unwinding.

```
// g++ / clang++ :  -fno-exceptions
// MSVC          :  remove /EHsc

throw std::runtime_error("x");   // with exceptions off -> std::terminate / c
```

Leaving the program

std::terminate

The runtime's last resort, triggered by an **unhandled exception** or a broken `noexcept`.

- Called **for** you, not by you
- Runs the current **terminate handler**
- The default handler calls **`std::abort`**
- No unwinding, no destructors

std::abort

Abnormal termination, immediately: no destructors, no flushing, raises `SIGABRT`.

```
#include <cstdlib>

std::abort();        // stops now; this is what a failed assert() calls
```

std::exit

Normal termination that still runs global cleanup, but skips **local** objects.

```
#include <cstdlib>

std::exit(EXIT_FAILURE);   // runs atexit handlers + static dtors, flushes st
```

return from main

The **clean** exit: unwinds locals, then destroys statics, like falling off the end of `main`.

```
int main() {
    Game game;              // ~Game() runs on return
    return EXIT_SUCCESS;
}
```

Which exit when?

Pick by **how much cleanup** you can still afford to run.

Mechanism	Local dtors	Static dtors	Use when
`return` from main	yes	yes	normal shutdown
`std::exit`	no	yes	bail out, flush logs
`std::abort`	no	no	unrecoverable bug
`std::terminate`	no	no	unhandled exception

Exceptions vs expected

Two answers to “this can fail”. Pick by **how often** failure happens.

Use `std::expected`	Use exceptions
failure is **routine**	failure is **truly rare**
hot paths, tight loops	startup, tooling, editors
exceptions are **disabled**	exceptions are **enabled**
error is part of the API	error is exceptional

noexcept

Promising not to throw

noexcept marks a function that will **never** let an exception escape.

```
void swap(Buffer& a, Buffer& b) noexcept;

Buffer(Buffer&& other) noexcept;     // moves should not throw
```

noexcept unlocks optimizations

The library checks it to choose **faster** code paths.

- `std::vector` growth **moves** elements only if the move is `noexcept`
- Otherwise it **copies** to stay exception-safe, which is slower
- Callers skip generating **unwinding** scaffolding around the call

Breaking the promise is fatal

Throw out of a `noexcept` function and the program is **terminated**. No catch can save it.

```
void update() noexcept {
    throw std::runtime_error("boom");   // calls std::terminate immediately
}
```

Where noexcept belongs

Mark the operations the standard library and your callers **expect** to be safe.

- **Move** constructors and move assignment
- **Destructors** (already `noexcept` by default)
- **swap** and simple accessors
- Functions that genuinely **cannot** fail

What if there was a way...

to catch the **bugs**, the states that should be *impossible*, before they ever ship?

Bugs: assert & contracts

Errors vs bugs, revisited

Different problems, different tools. Do not use one for the other.

Situation	Tool
file missing, packet malformed	`expected` / exceptions
index out of range (logic bug)	`assert` / contracts
broken function precondition	`assert` / contracts
recoverable runtime failure	error as a **value**

assert

From <cassert>: check a condition that you believe is **always** true.

```
#include <cassert>

Entity& get(EntityId id) {
    assert(id < entities.size() && "entity id out of range");
    return entities[id];
}
```

assert disappears in release

Define NDEBUG and every `assert` compiles to **nothing**.

```
// debug build:    assert(p ≠ nullptr);  → checked
// -DNDEBUG build:  assert(p ≠ nullptr);  → removed, zero cost
```

assert guidelines

Asserts document **invariants** for the developer, not error handling for the user.

- The condition must have **no side effects**, since it vanishes in release
- Check **preconditions**, **postconditions**, and **invariants**
- Never assert on **external** input: that is a real error
- A failed assert means a **bug to fix**, not a case to recover

C++26 contracts

A language-level successor to `assert`: **preconditions**, **postconditions**, and assertions.

```
int divide(int a, int b)
    pre(b != 0)                 // precondition
    post(r: r * b == a)         // postcondition, r names the result
{
    contract_assert(b != 0);  // in-body assertion
    return a / b;
}
```

Contracts vs assert

Contracts move the checks **into the function signature**, where callers can see them.

`assert`	C++26 contracts
body only	`pre` / `post` on the **API**
macro, `<cassert>`	language **keywords**
on/off via NDEBUG	**per-build** semantic
no result access	`post` can name the **result**

SEI INSTITUTE

Contract semantics

The build chooses **what happens** when a contract is violated, with no recompiling the source.

- **ignore**: not checked, like release `assert`
- **observe**: checked, logs, then **continues**
- **enforce**: checked, then **terminates**

SAE INSTITUTE

static_assert

Checking at compile time

`assert` and contracts run while the game is **running**; `static_assert` runs while it is **built**.

```
static_assert(sizeof(void*) == 8, "64-bit builds only");

// a false condition is a compile error: the program never links
```

Compile-time vs runtime

The condition must be a **constant expression**, so it costs **nothing** at runtime.

`assert` / contracts	`static_assert`
checked at **runtime**	checked at **compile time**
can vanish in release	never in the binary at all
any runtime condition	**constexpr** condition only
failure **terminates**	failure **fails the build**

static_assert in games

Lock down assumptions the engine relies on, so a violation **never compiles**.

```
struct NetPacket { uint16_t id; uint16_t len; uint32_t tick; };
static_assert(sizeof(NetPacket) == 8, "packet layout changed");

template< typename T >
void serialize(const T& v) {
    static_assert(std::is_trivially_copyable_v< T >, "T must be POD");
}
```

References

- cppreference: std::expected
- cppreference: std::optional
- cppreference: std::error_code
- cppreference: program support utilities
- cppreference: noexcept specifier
- P2900: Contracts for C++

Conclusion

- Separate **errors** (runtime, recoverable) from **bugs** (broken assumptions): they need different tools.
- **Error codes** and `std::error_code` are easy to ignore and split value from status.
- `std::optional` says *missing*; **`std::expected`** (C++23) returns a value **or** an error in one type, ignorable by no one.
- **Exceptions** keep the happy path clean but cost binary size and predictable timing, so many games disable them with `-fno-exceptions`.
- Know how a program **leaves**: `return` unwinds and cleans up, `std::exit` skips locals, `std::abort` / `std::terminate` stop cold.
- **`noexcept`** promises no throw, unlocks faster moves, and **terminates** if broken.
- Use **`assert`** / **C++26 contracts** for bugs (invariants and pre/postconditions), not for handling user-facing errors.
- **`static_assert`** moves the check to **compile time**: guard struct layouts and template requirements so a broken assumption never builds.